In [21]:
```
import numpy as np
import pandas as pd

# Training data (XOR problem)
X = np.array([
    [0, 0],
    [0, 1],
    [1, 0],
    [1, 1]
])

# Expected output
y = np.array([[0], [1], [1], [0]])

# Activation functions
def sigmoid(x):
    return 1 / (1 + np.exp(-x))

def sigmoid_derivative(x):
    return x * (1 - x)


# Initialize weights
np.random.seed(42)

W1 = np.random.rand(2, 2)
b1 = np.zeros((1, 2))

W2 = np.random.rand(2, 1)
b2 = np.zeros((1, 1))

learning_rate = 0.1
epochs = 10000

for epoch in range(epochs):

    # ---- Forward pass ----
    a1 = sigmoid(np.dot(X, W1) + b1)

    a2 = sigmoid(np.dot(a1, W2) + b2)

    # ---- Loss (MSE) ----
    loss = np.mean((y - a2) ** 2)

    # ---- Backward pass ----
    d_a2 = (a2 - y)
    d_z2 = d_a2 * sigmoid_derivative(a2)

    d_W2 = np.dot(a1.T, d_z2)
    d_b2 = np.sum(d_z2, axis=0, keepdims=True)

    d_a1 = np.dot(d_z2, W2.T)
    d_z1 = d_a1 * sigmoid_derivative(a1)

    d_W1 = np.dot(X.T, d_z1)
    d_b1 = np.sum(d_z1, axis=0, keepdims=True)

    # ---- Update weights ----
```

```
    W1 -= learning_rate * d_W1
    b1 -= learning_rate * d_b1
    W2 -= learning_rate * d_W2
    b2 -= learning_rate * d_b2

    if epoch % 1000 == 0:
        print(f"Epoch {epoch}, Loss: {loss:.4f}")

print("\nFinal Predictions:")
print(a2.round())
```

```
Epoch 0, Loss: 0.2521
Epoch 1000, Loss: 0.2476
Epoch 2000, Loss: 0.2277
Epoch 3000, Loss: 0.1743
Epoch 4000, Loss: 0.0697
Epoch 5000, Loss: 0.0199
Epoch 6000, Loss: 0.0098
Epoch 7000, Loss: 0.0063
Epoch 8000, Loss: 0.0045
Epoch 9000, Loss: 0.0035

Final Predictions:
[[0.]
 [1.]
 [1.]
 [0.]]
```